

目錄

1. [Java 開發環境準備](#) 概念:上課前要先裝好 Java 與 IntelliJ 這套 IDE(可以想成是「寫程式用的專業文書處理軟體」),並確認版本沒問題,這樣後面的範例才跑得起來。
2. [Spring 框架是什麼、為什麼需要它](#) 概念:說明 Spring 框架的賣點,以及沒有 Spring Boot 之前,傳統 Spring 專案光是「設定」就要寫一堆繁瑣的樣板程式碼。
3. [Spring Boot 怎麼解決傳統 Spring 的麻煩](#) 概念:Spring Boot 內建了一個「內嵌伺服器」,讓你不用額外裝 Tomcat、不用把專案包成 WAR 檔丟到外部伺服器,寫完直接執行就能跑網站。
4. [常見問題 Q&A、建立第一個專案與 Maven 建置工具](#) 概念:解答「Spring Boot 會不會取代 Spring MVC」「需不需要特定 IDE」這類新手疑問,並實際用 Spring Initializr(線上專案產生器)建出第一個專案、寫出第一支 REST API,順便認識 Maven 這個「自動幫你下載套件的採購員」。
5. [Spring Boot 專案結構長怎樣](#) 概念:介紹 Maven 標準的資料夾配置、application.properties 設定檔放哪裡,以及 Starter 套件包(一次幫你打包好一組相關依賴)的作用。
6. [Starter Parent:統一管理套件版本](#) 概念:pom.xml 裡繼承的 parent 設定,幫你統一決定每個套件該用哪個版本,避免自己手動指定版本號時互相衝突。
7. [DevTools:改完程式碼自動重啟](#) 概念:開發時最煩的就是改一行程式碼要手動重開伺服器,加上 DevTools 這個工具後,存檔就會自動幫你重啟應用程式。
8. [Actuator:監控端點與安全防護](#) 概念:裝上 Actuator 套件後,應用程式會自動多出健康檢查、應用程式資訊等一堆 /actuator 開頭的檢查端點,並教你用 Spring Security 把這些端點加上登入保護,避免任何人都能看到系統內部資訊。
9. [脫離 IDE:命令列執行與自訂設定屬性](#) 概念:示範怎麼用 mvnw 打包成 JAR 檔,直接在終端機用 java -jar 執行(不用開 IDE),並學會用 application.properties 搭配 @Value 註解讀取自己定義的設定值,認識 Spring Boot 上千個內建屬性大致怎麼分類。
10. [控制反轉\(IoC\)與依賴注入\(DI\)是什麼](#) 概念:物件不用自己手動 new 出來,而是交給 Spring 這個「總管」(Container)統一建立跟組裝,這個概念叫控制反轉;而「怎麼把需要的物件塞進去」的具體做法就是依賴注入。
11. [動手做:建構子注入實戰](#) 概念:實際寫一個教練建議 App,示範怎麼定義介面、標記 @Component 讓 Spring 認得這個物件,再透過建構子把依賴自動注入進 Controller。
12. [組件掃描:Spring 怎麼找到你寫的類別](#) 概念:Spring 預設只會掃描主程式所在套件底下的東西,如果你的類別放錯資料夾位置,Spring 就找不到它,這裡教你怎麼排查跟手動指定掃描範圍。
13. [Setter 注入、欄位注入,以及多個實作衝突時怎麼選](#) 概念:除了建構子,也可以用 Setter 方法或直接在欄位上標 @Autowired 來注入依賴;當同一個介面有好幾個實作類別讓 Spring 不知道選誰時,用 @Qualifier 指名要哪一個,或用 @Primary 設一個預設首選。
14. [@Lazy:讓 Bean 用到才建立](#) 概念:Spring 預設一啟動就把所有物件都建好,加上 @Lazy 可以改成「真正被呼叫到才建立」,說明這樣做的好處跟要注意的地方。

15. **Bean 作用域:共用一份還是每次都重做** 概念:解釋 Spring 建立的物件(Bean)預設是整個應用程式共用同一份(Singleton),還是每次要用都重新做一份(Prototype),以及怎麼切換這個行為。
16. **Bean 生命週期:出生到銷毀的掛勾方法** 概念:Bean 建立完成後、被銷毀前都可以掛一個自訂方法自動執行,適合拿來做初始化或釋放資源的動作。
17. **用 Java 程式碼手動配置 Bean** 概念:遇到別人寫好的類別(例如 AWS SDK)沒辦法加 @Component 註解時,改用 @Configuration 類別搭配 @Bean 方法,手動教 Spring 怎麼組出這個物件。
18. **Hibernate/JPA 概覽與 MySQL 資料庫安裝** 概念:認識 Hibernate 跟 JPA 這兩個「幫你把 Java 物件跟資料庫資料表對應起來」的技術(ORM),並動手安裝 MySQL、建好之後範例要用的資料表。
19. **接上資料庫:資料來源設定與 Entity 映射注解** 概念:設定資料庫連線帳密,用 CommandLineRunner 在程式一啟動時跑測試確認連得上,再用 @Entity、@Id、@Column 這些註解把 Java 類別「貼標籤」對應到資料庫表格跟欄位。
20. **DAO 模式:用 EntityManager 把物件存進資料庫** 概念:透過 DAO(資料存取物件,可以想成是專門負責跟資料庫溝通的窗口)搭配 EntityManager,實作新增單筆與批次新增多筆資料,並觀察主鍵自動遞增的行為。
21. **讀取物件:依主鍵查回單筆資料** 概念:用 entityManager.find() 依主鍵取回單一物件,並整理成 DAO 的讀取方法讓外部呼叫。
22. **查詢物件:認識 JPQL 查詢語言** 概念:JPQL 是專門針對 Java 物件(而不是資料表)寫的查詢語言,這裡示範基本語法跟具名參數的用法。
23. **排序、依條件查詢、更新與刪除** 概念:延續 DAO 加入依姓氏排序、依姓氏搜尋、更新單筆資料,以及刪除單筆或清空全部資料的方法。
24. **讓 Hibernate 自動建立資料表(ddl-auto)** 概念:不用自己手寫建表 SQL,讓 Hibernate 根據 Entity 類別自動生成資料表,並說明 ddl-auto 設定值(create/update/none 等)分別會不會把既有資料洗掉,正式環境該怎麼選才安全。
25. **REST API 是什麼:用天氣 App 舉例** 概念:用「手機 App 查天氣」的例子說明前後端為什麼可以用不同程式語言各自開發,只要靠 REST API 溝通就好,並複習 JSON 資料格式的基本寫法。
26. **HTTP 訊息結構、狀態碼與 Postman 工具** 概念:拆解一個 HTTP 請求跟回應長什麼樣子(標頭、狀態碼、內容類型),並介紹 Postman 這個工具,讓你不用寫前端也能手動測試 API。
27. **動手做出第一個 Spring REST Controller** 概念:建一個全新專案,加上 Web 依賴,寫一個標記 @RestController 的類別,實作 /hello 端點,再用 Postman 打打看驗證回傳結果。
28. **Jackson:Java 物件與 JSON 怎麼自動互轉** 概念:Spring 背後其實是靠 Jackson 這個套件,自動把 Java 物件轉成 JSON 回傳給前端,也能把前端傳來的 JSON 自動轉回 Java 物件,開發者完全不用手動寫轉換程式碼。
29. **學生資料 API 與路徑變數(Path Variable)** 概念:做出回傳學生清單的 API,並用路徑變數(例如網址 /students/1 裡的 1)做出查詢單一學生的端點,示範網址上的變數怎麼被 Spring 自動接進方法參數裡。
30. **例外處理:自訂錯誤訊息與全域統一管理** 概念:查詢不存在的學生 ID 時,與其丟出一坨看不懂的 500 錯誤,不如自訂例外類別跟錯誤格式,搭配 @ExceptionHandler 回傳乾淨的 404 訊息;後面再用 @ControllerAdvice 把這套錯誤處理邏輯抽出來,讓全部 Controller 共用同一套規則,不用每個 Controller 都寫一遍。

31. **怎樣算一個「好」的 REST API 設計** 概念:整理設計 API 的慣例(資源命名用名詞、用 HTTP 方法表達動作而不是塞在網址裡),並拿 PayPal、GitHub、Salesforce 的真實 API 當範例。
32. **即時專案第一步:員工資料表、Entity 與 DAO 查詢** 概念:開始做一個完整的「員工目錄」專案:設計資料庫表格、建立 Employee 這個 Entity 類別,並寫出用 EntityManager 取得員工清單的 DAO。
33. **加上 REST Controller 與 Service 分層架構** 概念:先把 DAO 包成對外的查詢 API,接著在 Controller 跟 DAO 中間多加一層 Service,變成「Controller 收發請求、Service 處理商業邏輯、DAO 管資料庫」的三層分工,並用 @Transactional 確保一連串資料庫操作要嘛全部成功要嘛全部復原。
34. **完成 CRUD:新增、更新、PATCH 局部更新與刪除 API** 概念:依序做出新增(POST)、整筆更新(PUT)、只改部分欄位的局部更新(PATCH),以及刪除(DELETE)員工的四個 API,並解釋為什麼 PUT 容易不小心把沒傳的欄位洗成空值,所以才需要 PATCH。
35. **Spring Data JPA:一個介面讓 DAO 大幅簡化** 概念:前面手寫的 DAO 其實有很多重複邏輯,這裡改用 Spring Data JPA 的 JpaRepository,只要宣告一個介面,連 findAll、save、deleteById 這些方法都不用自己寫,程式碼大幅減少。

1. Java 開發環境

開始學 Spring Boot 之前,課程假設你已經有 Java 底子,對 OOP、類別、介面、繼承、例外處理、集合這些基本功要熟。如果這些名詞聽起來還很陌生,建議先回頭補一下 Java 基礎,不然後面會很吃力。

環境準備只要兩樣東西:

- **JDK 17 以上**:Spring Boot 4 的最低要求是 JDK 17,版本太舊直接跑不起來。
- **一套 Java IDE**:課程用 IntelliJ,分免費的 **Community Edition**(下載自 jetbrains.com/idea/download,記得選 Community 版)跟付費的 **Ultimate Edition**(有試用期)。其實用哪套 IDE 都可以,IntelliJ 只是課程示範用的工具。

裝好之後不要急著往下,先寫一個最陽春的 Hello World Java 程式跑跑看。這一步的意義不是練習寫程式,而是「健檢」——確認 JDK 跟 IDE 真的裝對、能正常編譯執行,免得之後進到 Spring 階段才發現環境本身有問題,那時候要除錯會分不清是環境問題還是 Spring 的問題。

2. Spring 框架簡介

Spring 是 Java 世界裡最紅的框架之一,核心賣點是提供大量現成的輔助類別跟註解(annotation),讓你不用什麼都自己刻。

但傳統寫法 Spring 應用程式其實蠻痛苦的,難點大致三個:

1. **依賴要自己挑**:一個專案到底需要哪些 JAR 檔,常常要自己摸索。
2. **配置很雜**:要用 XML 還是 Java Config 來設定,選擇跟寫法都不輕鬆。
3. **伺服器要自己裝**:Tomcat、JBoss、WebSphere 這些容器,安裝、設定、部署都是額外工程。

這三個痛點,就是 Spring Boot 出現要解決的問題,也是下一節的主題。

3. Spring Boot 解決方案

Spring Boot 的核心價值一句話:讓 **Spring** 開發的起步變得超容易。它不是取代 Spring,而是在 Spring 之上做了一層「懶人包」,實際跑起來底層還是 Spring 的程式碼。可以想成 Spring 是食材跟廚具, Spring Boot 是幫你把常用套餐都配好、鍋子也預熱好的半成品包——你還是在煮 Spring 這道菜,只是省掉一堆前置作業。

具體做了三件事:

- **簡化配置**:根據你的 properties 檔跟 classpath 裡有什麼 JAR,自動幫你把該有的設定配好,這叫**自動配置 (auto-configuration)**。
- **依賴管理**:幫忙處理 Maven/Gradle 依賴衝突的問題。
- **內嵌式 HTTP 伺服器**:Tomcat、Jetty、Undertow 這些伺服器直接「內建」到你的應用程式裡,不用另外裝。

因為底層仍是 Spring,開發者還是得同時懂 Spring Boot 跟 Spring 本身的觀念。

Spring Initializr(start.spring.io)是官方提供的專案產生器:選好要用的依賴項,它就自動生出一個 Maven 或 Gradle 專案給你下載,解壓縮後匯入 IDE 即可,不限定 IDE,純文字編輯器搭配命令列跑 Maven 也行。

內嵌伺服器的實際效果是:打包出來只有「一個」JAR 檔(例如 **mycoolapp.jar**),裡面同時裝著你的程式碼跟一顆 Tomcat。這個 JAR 是自給自足的單元,不靠外部裝好的伺服器,可以直接:

```
> java -jar mycoolapp.jar
```

從 IDE 執行或命令列執行都可以,啟動同時應用程式跟伺服器就一起跑起來了。

值得注意的是, Spring Boot 並沒有把傳統部署方式鎖死。除了打成內嵌 JAR,也可以照舊打成 **WAR** 檔部署到企業內部既有的外部伺服器(Tomcat、JBoss、WebSphere)上——這種情況下 WAR 裡就不會包伺服器,因為外部伺服器本來就有了。同一台 Tomcat 上甚至可以同時掛好幾個團隊的 WAR(像 **travel.war**、**shopping.war**),互不干擾。

4. Spring Boot 常見問題 (FAQ)

幾個新手常見疑問先釐清:

- **Spring Boot 會取代 Spring MVC / Spring REST 嗎?** 不會,兩者不是競爭關係。Spring Boot 只是在背後把配置這件事簡化了,配置做完之後你寫的還是原汁原味的 Spring 程式碼。
- **執行速度會比較快嗎?** 不會。幕後跑的程式碼跟傳統 Spring 完全一樣, Spring Boot 的價值是讓你少寫配置、快速上手,而不是效能優化。
- **需要特定 IDE 嗎?** 不用,任何 IDE、甚至純文字編輯器都行。Spring 官方有出一套 **Spring Tools Suite(STS)**,本質是一組 IDE 插件,提供更精緻的 Spring 工具支援,但這是加分項不是必需品。

Maven 到底在幹嘛?

寫 Java 專案常常要用到別人寫好的 JAR(Spring、Hibernate、Commons Logging、JSON 處理套件等等)。沒有 Maven 的年代,你得自己一個一個跑到官網下載 JAR,再手動塞進 build path,超級繁瑣又容易漏掉相依的相依。

Maven 解決的方式,可以想成請了一位「個人採購員」:你只要開一張購物清單(告訴它你要用 Spring、Hibernate),它就自動連網幫你把對應的 JAR 抓下來,還會自動處理編譯期跟執行期該放哪裡,連 JAR 之間彼此依

賴的東西(比如 Spring 依賴 commons-logging)都會一併抓齊,完全不用你手動追。抓下來的東西會存一份在電腦本地的 **Maven Local Repository** 當快取,下次不用重複連網下載。

實際開發流程:

1. 到 Spring Initializr 網站配置專案(選 Maven/Gradle、Java 版本、Spring Boot 版本——記得選沒有 **SNAPSHOT** 字樣的正式發布版,SNAPSHOT 還在開發中不穩定)。
2. 下載產生的 **.zip**。
3. 解壓縮,建議搬到專門放開發專案的資料夾(例如 **dev-spring-boot**),不要留在下載資料夾裡亂。
4. 匯入 IDE,IDE 會自動開始下載 Maven 依賴,第一次匯入需要耐心等待。

配置頁面裡幾個關鍵欄位:**Group**(組織識別碼,慣例用反向網域,如 **com.luv2code**)、**Artifact**(專案本身的名字,最後產出的 JAR 就叫這個名字)、**Packaging**(選 Jar 或 War)、**Dependencies**(這裡選你需要的 Starter,例如選 **Web** 就會拿到完整的 Web 全端能力,含內嵌 Tomcat 跟 Spring MVC)。

Maven 標準目錄結構(這套結構所有主流 IDE 都認得,專案可以在不同 IDE 間自由搬家,不用為了「該用哪套 IDE」吵架):

目錄	用途
src/main/java	Java 原始碼
src/main/resources	屬性檔 / 設定檔
src/main/webapp	Web 資產(JSP、圖片、CSS——僅適用 WAR 封裝)
src/test	單元測試程式碼
target	Maven 編譯後產出的目錄

pom.xml(Project Object Model)是整個 Maven 專案的設定檔,就是前面說的那張「購物清單」,永遠放在專案根目錄,主要包含三塊:

```
<project ...>
  <groupId>com.luv2code</groupId>
  <artifactId>mycoolapp</artifactId>
  <version>1.0.FINAL</version>
  <packaging>jar</packaging>
  <dependencies>
    <!-- 列出所有依賴 -->
  </dependencies>
  <!-- plugins 用來做額外的自定義任務,例如自動產生測試報告 -->
</project>
```

其中 **groupId** + **artifactId** + **version** 合稱 **GAV**,是唯一識別一個專案的座標,可以類比成地址:Group ID 像城市、Artifact ID 像街道、Version 像門牌號碼。加依賴時版本雖然技術上可省略,但強烈建議寫死,才能保證每次建置結果一致、也方便追蹤測試過的是哪個版本。

要查某個依賴的正確座標,一種是去該專案官網找,更推薦的做法是直接上 **Maven Central Repository**(**central.sonatype.com**)搜尋——尤其專案動輒要管理十幾二十個依賴時,一站找齊比逐一跑官網有效率太多。

5. Spring Boot 專案結構

透過 Spring Initializr 產生出來的其實就是一個標準 Maven 專案,目錄結構跟第 4 節提到的一致 (`src/main/java` 放程式碼、`src/main/resources` 放設定檔、`src/test/java` 放測試)。

專案裡會多兩個檔案:`mvnw` 與 `mvnw.cmd`,叫 **Maven Wrapper**。它的好處是電腦不用預先裝 Maven、也不用設 PATH,執行時如果找不到對的 Maven 版本,它會自動幫你下載——等於把 Maven 本身也「內嵌」進專案了。用法依系統而定:

```
# Windows
mvnw clean compile test

# Linux / Mac
./mvnw clean compile test
```

如果電腦本來就裝好 Maven,`mvnw` 系列檔案可以直接刪掉不管,改用一般的 `mvn clean compile test` 就好。

`pom.xml` 會自動帶入你在 Initializr 網站填的 GAV 資訊。而 **Spring Boot Starters** 則是這章的重點觀念:一組已經包裝好、確保版本相容的 Maven 依賴集合,目的是省掉你自己去湊齊一堆零散依賴的麻煩。例如加一個 `spring-boot-starter-web`,實際上就一次拿到 Spring Web MVC、內嵌 Tomcat、JSON 處理工具——一站買齊,不用自己一樣一樣挑。

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

`pom.xml` 底部 `<build>` 區塊裡的 **Spring Boot Maven plugin**,負責把專案打包成可執行的 JAR/WAR,也能直接跑起來:

```
./mvnw package          # 打包
./mvnw spring-boot:run  # 直接執行
```

application.properties 是 Spring Boot 預設會自動讀取的設定檔,位置固定在 `src/main/resources/application.properties`。改個伺服器 port 只要一行:

```
server.port=8585
```

也可以自己定義任意屬性,再用 `@Value` 註解配合 `${...}` 語法把值「注入」到欄位裡,不用自己寫程式碼去解析檔案:

```
coach.name=Mickey Mouse
team.name=The Mouse Crew
```

```
@RestController
public class FunRestController {
    @Value("${coach.name}")
    private String coachName;

    @Value("${team.name}")
    private String teamName;
}
```

有兩個目錄要分清楚,踩雷率很高:靜態資源(HTML、CSS、JS、圖片等)預設要放在 `src/main/resources/static`;模板檔(FreeMarker、Thymeleaf、Mustache, Spring Boot 都內建自動配置)要放在 `src/main/resources/templates`。切記:如果打包成 **JAR**,絕對不要用 `src/main/webapp`,那個目錄是 Maven 給 **WAR** 封裝用的標準目錄,JAR 封裝的建置工具會直接忽略它。此外 Spring Initializr 也會自動生成一個基礎測試類別(如 `MycoolappApplicationTests.java`),可以直接在裡面擴充自己的單元測試。

6. Spring Boot Starter Parent

這是 Spring Boot 提供的一個特殊 Starter,作用不是加功能,而是幫你把 **Maven** 的一堆預設值先定好——像編譯器層級、UTF-8 原始碼編碼這些每個專案都要設一次的東西,直接繼承現成的就好。用 Spring Initializr 產生專案時,`pom.xml` 會自動幫你掛上:

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.0.0</version>
  <relativePath><!-- lookup parent from repository --></relativePath>
</parent>
```

想覆寫某個預設值(例如指定 Java 版本),在 `<properties>` 裡設定對應 property 即可:

```
<properties>
  <java.version>25</java.version>
</properties>
```

它帶來最實用的好處是**版本管理**:凡是 `spring-boot-starter-*` 開頭的依賴,`<dependency>` 裡完全不用寫 `<version>`,版本會自動從 parent 繼承,既省事又能保證彼此相容,不會因為手動填錯版本號而互相打架:

```
<dependencies>
  <dependency>
```

```
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
</dependencies>
```

另外它也順帶把 **Spring Boot Maven Plugin** 的預設設定準備好了,<build> 裡直接引用即可,不用再額外配置,接著就能一行指令跑起應用程式:

```
> mvn spring-boot:run
```

7. Spring Boot DevTools

開發時最煩的事之一:改一行程式碼,就得手動停掉、重新啟動應用程式才看得到效果。**Spring Boot DevTools** 就是解決這個摩擦的工具——只要在 `pom.xml` 加一個依賴,完全不用寫任何額外程式碼,程式碼一存檔它就自動幫你重啟:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
</dependency>
```

多數 IDE 裝了就能直接用,但 **IntelliJ Community Edition** 是例外,需要手動打開兩個開關才會生效:

1. **Preferences** → **Build, Execution, Deployment** → **Compiler**,勾選 **Build project automatically**。
2. **Preferences** → **Advanced Settings**,勾選 **Allow auto-make to start even if developed application is currently running**。

這兩個設定不勾,DevTools 裝了也是白裝,IntelliJ 不會在背景幫你重新編譯。

實測流程很單純:啟動應用程式後,在 `FunRestController` 加一個新端點,存檔:

```
@GetMapping("/workout")
public String getDailyWorkout() {
    return "Run a hard 5k!";
}
```

存檔後觀察 Console,會看到類似這樣的訊息,代表 DevTools 已經偵測到變更並自動重啟完成:

```
LiveReload server is running on port 35729
Tomcat started on port(s): 8080 (http) with context path ''
```

不用手動重啟,直接重新整理瀏覽器就能看到新端點生效,反覆改個幾次(課程裡又追加了 `/fortune` 端點驗證)都一樣穩定,證實這套自動重載機制值得信賴,是實務開發中提升效率的標配工具。

8. Spring Boot Actuator

應用程式上線之後,維運會有一堆問題要問:現在運作正常嗎?健康狀態如何?有沒有效能指標可以看?**Spring Boot Actuator** 就是專門回答這些問題的模組——只要加一個依賴,不用寫程式碼,就能「免費」拿到一整批監控與管理用的 REST 端點,全部掛在 `/actuator` 這個前綴底下:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

預設只暴露 `/health`,存取回傳很簡單的 JSON:

```
{ "status": "UP" }
```

`/info` 端點預設是空的(`{}`),要讓它有內容,得先開放暴露、再自己填資料。開放端點要改 `application.properties`:

```
management.endpoints.web.exposure.include=health,info
management.info.env.enabled=true
```

`/info` 的內容規則很簡單:凡是 `info.` 開頭的屬性都會被收進 JSON:

```
info.app.name=My Super Cool App
info.app.description=A crazy and fun app, yohoo!
info.app.version=1.0.0
```

```
{ "app": { "name": "My Super Cool App", "description": "A crazy and fun
app, yohoo!", "version": "1.0.0" } }
```

Actuator 其實有 20 幾個端點,比較常用的還有:`/beans`(列出應用程式裡所有註冊的 Spring bean,除錯配置很好用,可以確認某個 bean 到底有沒有被建立)、`/threaddump`(執行緒快照,拿來分析效能瓶頸)、`/mappings`(列出所有請求路由對應)、`/audit-events`(審計事件)。想一次全開,用萬用字元:

```
management.endpoints.web.exposure.include=*
```

要注意的是,這些端點會洩漏應用程式內部細節,直接開放給任何人存取是安全隱患。解法是加上 **Spring Security**:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

加了之後存取受保護端點會跳出登入視窗,預設帳號是 **user**,密碼是啟動時 Console 印出的一長串隨機字串 (Using generated security password: ...)。這組密碼只適合開發測試,正式環境要自己指定固定帳號:

```
spring.security.user.name=scott
spring.security.user.password=tiger
```

如果只是不想暴露特定端點(而不是整個關掉安全機制),可以用 **exclude**:

```
management.endpoints.web.exposure.exclude=health,info
```

排除之後再存取會變成 404 Whitelabel Error Page。不過要提醒:實務上 **/health**、**/info** 這兩個端點對 DevOps 監控很有價值,通常建議保留,只有教學或實驗目的才會臨時排除,用完記得註解掉(**#**)還原。

9. 從命令列執行 Spring Boot

開發時多半掛在 IDE 裡跑,但因為伺服器已經內嵌在 JAR 裡,Spring Boot 應用程式其實可以完全脫離 IDE、單靠命令列獨立運行——這對之後要部署到伺服器或寫自動化腳本很重要。

兩種命令列啟動方式:

```
# 方式一:直接執行打包好的 JAR(內建程式碼 + 內嵌伺服器一起啟動)
java -jar mycoolapp.jar

# 方式二:透過 Spring Boot Maven plugin,不用先找 JAR 路徑
./mvnw spring-boot:run
```

打包指令是 **./mvnw package**(或已裝 Maven 的話用 **mvn package**),看到 **BUILD SUCCESS** 代表打包完成,JAR 會生在 **target/** 目錄下(例如 **target/mycoolapp-0.0.1-SNAPSHOT.jar**)。停掉正在跑的程式用 **CTRL+C**。

mvnw / mvnw.cmd 這對 Maven Wrapper 檔案的重點是:電腦不用先裝 Maven,執行時找不到對的版本它會自動下載,也確保團隊每個人用的 Maven 版本一致。Windows 用 **mvnw.cmd**(或直接 **mvnw**),Mac/Linux 用 **./mvnw**。

Windows 跟 macOS/Linux 走的流程幾乎一樣,幾個容易忽略的細節:

- **路徑不要有空格**:目錄名稱裡有空格,Java runtime 有時候會處理不了。
- 先確認 Java 環境正確:**echo %JAVA_HOME%**(Windows)跟 **java --version**,兩者都要有正確輸出。
- **cd** 到專案目錄後執行 **./mvnw package**,再 **cd target** 用 **java -jar <檔名>.jar** 手動啟動,或直接回專案根目錄用 **./mvnw spring-boot:run** 省事啟動。
- 驗證方式就是瀏覽器打對應 URL(例如 **/workout**、**/fortune**),看得到回傳內容就代表伺服器真的在跑;**CTRL+C** 停掉後,瀏覽器再訪問會變成 **ERR_CONNECTION_REFUSED**,這正好證明伺服器狀態跟命令列是綁在一起的。

避免硬編碼(hard-coding) 是這節後半的重點。與其把數值寫死在程式碼裡,應該丟到 **application.properties**,再用 **@Value** 注入:

```
coach.name=Mickey Mouse
team.name=The Mouse Club
```

```
@Value("${coach.name}")
private String coachName;

@Value("${team.name}")
private String teamName;

@GetMapping("/teaminfo")
public String getTeamInfo() {
    return "Coach: " + coachName + ", Team name: " + teamName;
}
```

常見陷阱:複製貼上時漏掉花括號,寫成 **@Value("\${coach.name}")**(少了 **}**),Spring 不會報錯,但也不會幫你解析,瀏覽器只會原封不動印出 **\${coach.name}** 這串字面文字而不是實際值。務必確認語法是完整的 **\${property.name}**。改完存檔後 DevTools 一樣會自動重啟,不用手動重來。

Spring Boot 內建超過 1,000 個屬性,可依功能分成幾大類(Core、Web、Security、Data、Actuator、Integration、DevTools、Testing 等),不用死背,知道分類邏輯之後查找會快很多。幾個開發常用的:

```
# 日誌層級,依套件路徑設定,套用到子套件;層級由細到粗:TRACE < DEBUG < INFO < WARN <
ERROR < FATAL < OFF
logging.level.org.springframework=DEBUG
logging.level.com.luv2code=INFO
logging.file.name=my-crazy-stuff.log

# 伺服器連接埠與上下文路徑
server.port=7070
server.servlet.context-path=/mycoolapp
```

```
server.servlet.session.timeout=15m

# Actuator 基礎路徑(預設 /actuator)
management.endpoints.web.base-path=/my-actuator

# Spring Security 預設帳密
spring.security.user.name=admin
spring.security.user.password=topsecret

# 資料庫連線
spring.datasource.url=jdbc:mysql://localhost:3306/ecommerce
spring.datasource.username=scott
spring.datasource.password=tiger
```

值得特別注意 `server.servlet.context-path` 這個設定:一旦設了(例如 `/mycoolapp`),所有端點的 URL 都要加上這個前綴才能存取,原本沒加前綴的舊網址會直接變 404。同理改了 `server.port` 之後,舊的 port 也會連不上(`ERR_CONNECTION_REFUSED`)。這兩個是實測時最容易卡住的地方,改完設定記得同步更新瀏覽器網址,例如從 `localhost:8080/teaminfo` 變成 `http://localhost:7070/mycoolapp/teaminfo`。完整的常用屬性清單可以查官方文件(課程提供的捷徑是 www.luv2code.com/spring-boot-props)。

10. 控制反轉 (Inversion of Control, IoC)

IoC 講的是「物件怎麼被生出來、被管理」這件事,主控權要交給誰。以前的寫法是你自己在程式裡 `new` 出物件,現在改成把這個工作外包出去,交給 Spring 容器負責。可以想成:你不用自己去菜市場買菜、洗切備料,而是有人(Spring)幫你把食材都準備好,你只要說「我要一份 Coach」,它就把煮好的菜端給你。

Spring Container 的角色很像一座「物件工廠」。應用程式不會自己造物件,而是跟工廠說「給我一個 Coach 物件」,工廠會依照事先設定好的配置決定要組裝哪一種教練(板球教練、棒球教練……),組好之後把參考(reference)丟回來給你。這樣的好處是,今天想把板球教練換成棒球教練,完全不用動應用程式本身的程式碼,只要改配置就好——這就是「可配置性」。

Spring Container 兩大核心功能:

- 建立並管理物件 → 靠 IoC
- 注入物件的依賴項 → 靠依賴注入 (DI)

配置容器的方式有三種:XML 配置(舊,現在少用)、Java 註解(現代主流)、Java 原始碼(現代)。

依賴注入 (DI) 則是 IoC 更具體的實踐,核心精神是「依賴反轉原則」——客戶端不用自己準備好所有需要的東西,而是委託別人把這些東西準備好送過來。就像汽車工廠:客人只要說「給我一台車」,工廠內部處理引擎、車門、擋風玻璃這些零件的組裝,最後把完整的車交給客人,客人完全不用知道零件怎麼湊起來的。複雜一點的例子是「總教練」,他底下可能還帶了助理教練、體能教練、醫療團隊,Spring 在注入總教練時,會把這整條依賴鏈都準備好一起交給你。

Spring 官方最推薦的兩種注入方式:

方式	使用時機
建構子注入 (Constructor Injection)	依賴項是必要的,官方首選
Setter 注入 (Setter Injection)	依賴項是選配的,沒提供也能有預設邏輯

Spring 判斷要注入誰,靠的是**自動裝配 (AutoWiring)**——依「型別」去找符合的類別或介面。

實作建構子注入的標準流程:

1. 定義依賴的介面與實作類別
2. 建立 REST Controller
3. 在類別中寫一個建構子用來接收注入
4. 加上 `@GetMapping` 對外開放端點

```
public interface Coach {
    String getDailyWorkout();
}

@Component
public class CricketCoach implements Coach {
    @Override
    public String getDailyWorkout() {
        return "Practice fast bowling for 15 minutes";
    }
}

@RestController
public class DemoController {

    private Coach myCoach;

    @Autowired
    public DemoController(Coach theCoach) {
        myCoach = theCoach;
    }

    @GetMapping("/dailyworkout")
    public String getDailyWorkout() {
        return myCoach.getDailyWorkout();
    }
}
```

`@Component` 的作用就是跟 Spring 說「這個類別是一個 Spring Bean」,讓它有資格被納入依賴注入的候選名單。Spring Bean 說穿了就是一個「被 Spring 容器管理的一般 Java 類別」。

有個小知識點:如果一個類別只有一個建構子,那個建構子上的 `@Autowired` 其實可以省略不寫,Spring 會自動辨識並注入,只是保留下來對學習/閱讀比較清楚。

還有一個常見的 IDE 誤會:因為我們習慣「針對介面寫程式」(引用 `Coach` 而不是 `CricketCoach`),IDE 的靜態分析工具看不出 `CricketCoach` 在哪裡被用到,常常顯示 `no usages` 的警告。這其實是假警報,因為注入是在執行期由 Spring 動態完成的,IDE 掃不到而已,只要程式能正常跑出結果就不用理它。

最後一個常見疑問:「這麼簡單的事,我自己 `new` 不就好了,為什麼要用 Spring?」——確實對小專案來說,IoC/DI 帶來的好處不明顯。但 Spring 是為**企業級應用**設計的,除了 IoC/DI 之外,它還包了資料庫存取、事務處理、REST API、Web MVC、安全性等一整套生態系,真正的威力在處理大型複雜系統時才會顯現。

11. 使用 Spring Initializr 初始化專案

Spring Initializr (start.spring.io) 就像是專案的「出生證明產生器」, 幫你把骨架、依賴都設定好, 不用自己手刻 Maven 檔案。

建立專案時要填的東西:

- **Project:** 選 **Maven**
- **Language:** 選 **Java**
- **Spring Boot Version:** 選最新的正式發佈版, 避開帶 **SNAPSHOT** 字樣的版本 (那是還沒穩定的開發版)
- **Group / Artifact / Description:** 專案的身分資訊, Package name 會依 Group + Artifact 自動組出來
- **Packaging:** 選 **Jar**
- **Java Version:** 依需求選

常用依賴項可以直接搜關鍵字加進去, 例如輸入 **dev** 找到 **Spring Boot DevTools** (開發時存檔自動重啟)、輸入 **web** 找到 **Spring Web**。

設定好按下 **GENERATE**, 會下載一個 zip 檔, 解壓縮後把資料夾丟進你的專案根目錄, 重新命名成有意義的名字。因為它是標準 Maven 專案, 可以直接把資料夾拖進 IntelliJ IDEA 開啟, IDE 會自動做「Scanning Maven projects...」「Downloading dependencies...」這類同步動作, 一定要等它跑完才能開始寫程式。

接著就是動手建立 **Coach** 介面、**CricketCoach** 實作 (記得加 **@Component**)、**DemoController** (**@RestController**), 流程跟第 10 節一致。

這節比較實用的是一個**踩雷排除**: 如果改了程式碼儲存後, DevTools 沒有自動重啟, 通常是 IntelliJ 的自動編譯沒開。要檢查兩個地方:

1. **Settings** → **Advanced Settings**, 勾選 **Allow auto-make even if developed application is currently running**
2. **Settings** → **Build, Execution, Deployment** → **Compiler**, 勾選 **Build project automatically**

兩個都打勾之後, 改字串內容存檔, 瀏覽器重新整理就能看到新結果, 不用手動重啟伺服器, 開發效率會差很多。

12. 組件掃描 (Component Scanning)

組件掃描是 Spring 用來「找 Bean」的機制: 它會自動掃描 Java 類別, 只要看到 **@Component** 之類的註解, 就自動把這個類別註冊成容器裡的 Bean。你可以想成 Spring 拿著一份「有貼標籤的東西全部帶回家」的清單在專案裡巡邏。

主程式類別 (例如 **SpringcoredemoApplication.java**) 上的 **@SpringBootApplication** 其實是三個註解包在一起的「複合註解」:

註解	功能
@EnableAutoConfiguration	啟用 Spring Boot 自動配置
@ComponentScan	掃描目前套件與所有子套件
@Configuration	允許用 @Bean 註冊額外 Bean、匯入其他配置類別

```
@SpringBootApplication
public class SpringcoredemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(SpringcoredemoApplication.class, args);
    }
}
```

執行 `SpringApplication.run()` 時, Spring Boot 會在背後建立 Application Context、註冊所有偵測到的 Bean、啟動內嵌伺服器 (如 Tomcat)。

關鍵陷阱:預設的組件掃描只會涵蓋「主啟動類別所在的套件」以及它底下所有的子套件 (遞迴), 範圍以外的平行套件是掃不到的。例如主程式在 `com.luv2code.springcoredemo`, 底下開 `controller`、`service`、`repository` 這些子套件都沒問題; 但如果組件放在 `com.luv2code.demo.utils` (跟主套件是平行關係、不是子套件), Spring Boot 完全不會理它, 啟動時就會丟出類似:

```
Parameter 0 of constructor in ... required a bean of type
'com.luv2code.util.Coach' but could not be found.
```

解法是在 `@SpringBootApplication` 上用 `scanBasePackages` 屬性, 把所有想被掃描的套件都明確列出來 (用逗號分隔):

```
@SpringBootApplication(scanBasePackages={
    "com.luv2code.util",
    "org.acme.cart",
    "edu.cmu.srs"
})
public class SpringcoredemoApplication { ... }
```

不過這只是「不得已時的補救」。實務上比較建議的做法是: 把所有 Controller、Service、Repository 等組件都放在主套件底下的子套件裡, 乖乖用預設掃描, 不用維護一份容易忘記更新的 `scanBasePackages` 清單。只有整合第三方套件或做多模組專案、組件真的散落在主套件外時, 才考慮手動指定範圍。

13. Setter 注入 (Setter Injection)

Setter 注入跟建構子注入是兩種官方推薦的注入方式, 差別在於它是透過呼叫類別裡的 setter 方法來把依賴塞進去, 而不是靠建構子。

```
@RestController
public class DemoController {

    private Coach myCoach;

    @Autowired
    public void setCoach(Coach theCoach) {
        myCoach = theCoach;
    }
}
```

```
    }
}
```

Spring 幕後流程是:先實例化依賴 (例如 `CricketCoach`)→ 再實例化目標類別 (`DemoController`)→ 呼叫該 setter 方法把依賴傳進去。

有個容易被忽略的彈性:注入其實不限定要用 `set` 開頭命名的方法,只要方法上有 `@Autowired`,隨便取名 (像 `doSomeStuff`)Spring 一樣認得,靠的是註解不是方法名稱。不過為了程式碼可讀性,實務上還是建議乖乖照 setter 命名慣例來寫。

三種注入方式的選用建議 (重要比較):

注入方式	適用情境	官方態度
建構子注入	依賴是必要的	首選、推薦
Setter 注入	依賴是選配的,沒有也能有預設邏輯	推薦
欄位注入 (Field Injection)	直接在 <code>private</code> 欄位上加 <code>@Autowired</code> ,靠 Java Reflection 塞值	不推薦,早期很流行但現在因為難以單元測試而被官方勸退

```
// 欄位注入範例(不推薦,但舊專案很常見)
@RestController
public class DemoController {
    @Autowired
    private Coach myCoach;
}
```

多個實作造成的歧義問題是這節的重頭戲。如果 `Coach` 介面同時有 `CricketCoach`、`BaseballCoach`、`TrackCoach`、`TennisCoach` 四個 `@Component` 實作,Spring 會因為不知道選誰而啟動失敗,錯誤訊息類似:

```
required a single bean, but 4 were found: baseballCoach, cricketCoach,
tennisCoach, trackCoach
```

(順帶一提,Bean ID 預設規則是類別名稱第一個字母變小寫,`CricketCoach` → `cricketCoach`。)

解法有兩種,設計理念不同,務必分清楚:

- `@Qualifier("cricketCoach")`:在注入點「精確指定」要哪一個,由使用端 (呼叫方)決定

```
@Autowired
public DemoController(@Qualifier("cricketCoach") Coach theCoach) {
    myCoach = theCoach;
}
```

- `@Primary`:在實作類別上標記「我是預設的那個」,把決定權交給實作端本身

```
@Component
@Primary
public class TrackCoach implements Coach { ... }
```

兩者的優先順序:**@Qualifier** 優先於 **@Primary**——就算某個類別被標成 **@Primary**,只要注入點寫了 **@Qualifier** 指定別人, Spring 還是聽 **@Qualifier** 的。官方建議優先用 **@Qualifier**,因為它更精確、優先權也更高。另外要注意,**@Primary** 只能標記在一個類別上,如果同時有兩個類別都標 **@Primary**,一樣會啟動失敗 (more than one 'primary' bean found among candidates)。

14. 延遲初始化 (Lazy Initialization)

Spring 預設的行為是:應用程式一啟動,所有標了 **@Component** 之類註解的 Bean 就會**全部**被建立好、放著待命——即使暫時用不到。這在 Bean 很多的大型專案裡,會拖慢啟動速度。

延遲初始化就是反過來:讓某個 Bean 「不要一開始就急著出現」,等到真的被需要 (被依賴注入或被明確請求) 時才建立。這有點像餐廳不用一開始就把所有菜都先做好放著,而是客人點了才現做。

單一類別做法,加 **@Lazy**:

```
@Component
@Lazy
public class TrackCoach implements Coach {
    public TrackCoach() {
        System.out.println("In constructor: " +
            getClass().getSimpleName());
    }
    ...
}
```

加了之後,如果 **TrackCoach** 沒有被任何地方注入或請求,啟動時的 log 就不會出現它的建構子訊息,其他沒標 **@Lazy** 的 (**BaseballCoach**、**CricketCoach**、**TennisCoach**)還是照常在啟動時建立。

如果想讓「全部」Bean 都變成延遲初始化,不用一個個加 **@Lazy**,直接在 **application.properties** 開全域開關:

```
spring.main.lazy-initialization=true
```

開了之後,連 **DemoController** 都不會在啟動時建立,要等第一次有請求打進來 (例如訪問 **/dailyworkout**) 才觸發依賴解析:先建 **CricketCoach**,再建 **DemoController** 並把前者注入進去,順序在 log 裡可以清楚看到。

優缺點要記清楚 (這是判斷要不要用的關鍵):

內容

優點

只在真正需要時才建立物件;Bean 數量龐大時可以加快啟動速度

缺點

Web 組件 (如 `@RestController`)要等第一次被請求才建立,第一次回應會變慢;配置錯誤可能要等真正用到才會爆出來,不會在啟動時就發現;要確保所有 Bean 陸續建立完後,記憶體仍然夠用

官方態度是:延遲初始化預設是關閉的,而且建議不要為了「感覺上」比較快就貿然開啟——這是典型的「過早優化」陷阱,應該先做效能分析 (profiling),確認啟動速度真的是瓶頸,再決定要不要用。

15. Bean 作用域 (Bean Scopes)

作用域 (Scope)講的是 Bean 的生命週期規則:它能活多久、會建立幾個實例、怎麼被共享。可以想成「這個物件是大家共用同一份,還是每個人都各拿一份新的」。

Singleton 是預設作用域:整個容器只會建立唯一一個實例,快取在記憶體裡,不管有多少地方注入它,拿到的都是同一個物件。就像公司裡的印表機,大家都是共用同一台。

```
@Component
@Scope(ConfigurableBeanFactory.SCOPE_SINGLETON) // 顯式寫出來(其實是預設值)
public class CricketCoach implements Coach { ... }
```

除了 Singleton, Spring 還提供:

作用域	說明
singleton	單一共享實例(預設)
prototype	每次容器請求/每個注入點都建立一個全新實例
request	限定於單次 HTTP 請求(僅 Web 應用)
session	限定於單次 HTTP Session(僅 Web 應用)
application	限定於 ServletContext(僅 Web 應用)
websocket	限定於 WebSocket(僅 Web 應用)

Prototype 跟 Singleton 完全相反:每次有新的注入點需要這個 Bean, Spring 都會重新做一個新的給你,彼此互不影響。像便利商店的杯子,每個人都拿一個全新的,不會共用。

```
@Component
@Scope("prototype") // 或 @Scope(ConfigurableBeanFactory.SCOPE_PROTOTYPE)
public class CricketCoach implements Coach { ... }
```

驗證作用域的常見小技巧,是同時注入兩個依賴,用 `==` 比較是不是同一個物件:

```
@GetMapping("/check")
public String check() {
    return "Comparing beans: " + (myCoach == anotherCoach);
}
```

- Singleton → 回傳 **true** (同一個實例)
- Prototype → 回傳 **false** (兩個不同實例)

實務上大多數 Bean 用預設的 Singleton 就好 (省資源),只有在需要「每個使用場景都要有獨立、不互相干擾的狀態」時,才考慮用 Prototype。

16. Bean 生命週期方法

Bean 從被建立到被銷毀,會走過一條固定的路徑,Spring 允許你在中間的關鍵時間點「插一腳」,塞入自己的初始化或清理邏輯:

```
容器啟動 → Bean 被實例化 → 依賴被注入 → Spring 內部處理
          → 【你的自訂初始化方法】 → Bean 可以使用了
          → 容器關閉 → 【你的自訂銷毀方法】 → 結束
```

這兩個「插旗點」用兩個註解就能搞定,不用實作任何介面:

- **@PostConstruct**: Bean 建構完成、依賴也注入好之後,自動執行。適合拿來設定資料庫連線、Socket、檔案控制代碼這類「開機準備」的工作。
- **@PreDestroy**: 容器關閉、Bean 要被銷毀之前,自動執行。適合拿來做資源清理、關閉連線這類「收尾」工作。

```
@Component
public class CricketCoach implements Coach {

    public CricketCoach() {
        System.out.println("In constructor: " +
            getClass().getSimpleName());
    }

    @PostConstruct
    public void doMyStartupStuff() {
        System.out.println("In doMyStartupStuff: " +
            getClass().getSimpleName());
    }

    @PreDestroy
    public void doMyCleanupStuff() {
        System.out.println("In doMyCleanupStuff: " +
            getClass().getSimpleName());
    }
}
```

```
}  
}
```

方法名稱可以自己取 (`doMyStartupStuff`、`doMyCleanupStuff` 都只是範例), Spring 認的是註解不是名字。實際跑起來, 啟動、關閉應用程式時 console 會依序印出:

```
In constructor: CricketCoach  
In doMyStartupStuff: CricketCoach  
In doMyCleanupStuff: CricketCoach
```

清楚驗證了「建構子 → `@PostConstruct` → (使用中) → `@PreDestroy`」這條時間軸。開發步驟很簡單, 就是先在類別裡寫好兩個方法, 再分別貼上 `@PostConstruct` 和 `@PreDestroy`。

17. 使用 Java 程式碼配置 Bean (Configuring Beans with Java Code)

除了靠 `@Component` 讓組件掃描自動抓, 還有第二種註冊 Bean 的方式: 完全用 Java 程式碼手動配置, 不用在類別上加任何註解。

開發流程三步驟:

```
Step 1: 建立 @Configuration 類別  
Step 2: 用 @Bean 方法定義 Bean  
Step 3: 把 Bean 注入到需要的地方
```

步驟一, 建立一個標了 `@Configuration` 的類別, 作為 Spring 的配置來源:

```
@Configuration  
public class SportConfig {  
}
```

步驟二, 在裡面寫一個 `@Bean` 方法, 手動 `new` 出物件再回傳。Bean 的 ID 預設就是方法名稱(這裡是 `swimCoach`):

```
@Configuration  
public class SportConfig {  
  
    @Bean  
    public Coach swimCoach() {  
        return new SwimCoach();  
    }  
}
```

步驟三,在需要的地方用 `@Autowired + @Qualifier("swimCoach")` 把它注入進來,跟一般的 Bean 用法完全一樣。

如果不想用方法名稱當 ID,可以自己指定:

```
@Bean("aquatic")
public Coach swimCoach() {
    return new SwimCoach();
}
```

之後注入時就改用 `@Qualifier("aquatic")`。

為什麼不乾脆全部用 `@Component` 就好,幹嘛還要 `@Bean`? 關鍵差別在於:`@Component` 要求你能改到那個類別的原始碼、親手貼上註解;但如果那個類別是**第三方套件** (例如只有一份 `.jar` 檔,原始碼你根本碰不到、也不該去改),`@Component` 這條路就走不通了。這時候 `@Bean` 就是唯一的解法——在自己的 `@Configuration` 類別裡手動 `new` 出這個第三方物件,把它包裝成 Spring Bean。

典型案例是整合 AWS S3(雲端檔案儲存服務)的 client:

```
@Configuration
public class DocumentsConfig {

    @Bean
    public S3Client remoteClient() {
        ProfileCredentialsProvider credentialsProvider =
        ProfileCredentialsProvider.create();
        Region region = Region.US_EAST_1;

        return S3Client.builder()
            .region(region)
            .credentialsProvider(credentialsProvider)
            .build();
    }
}
```

配置好之後,`S3Client` 就跟一般 Spring 組件一樣可以直接 `@Autowired` 注入到別的 Service 裡使用:

```
@Component
public class DocumentService {

    private S3Client s3Client;

    @Autowired
    public DocumentService(S3Client theS3Client) {
        s3Client = theS3Client;
    }

    public void processDocument(Document theDocument) {
```

```
PutObjectRequest putObjectRequest = PutObjectRequest.builder()
    .bucket(bucketName)
    .key(subDirectory + "/" + fileName)
    .acl(ObjectCannedACL.BUCKET_OWNER_FULL_CONTROL)
    .build();

s3Client.putObject(putObjectRequest,
    RequestBody.fromInputStream(fileInputStream, contentLength));
}
```

一句話總結: `@Component` 是「我自己寫的類別,直接貼標籤讓 Spring 掃到」; `@Bean` 是「別人寫好的東西,我用一個工廠方法把它包裝、塞進 Spring 容器」。兩者殊途同歸,最後都變成可以 `@Autowired` 的 Spring Bean。

18. Hibernate / JPA 概覽

Hibernate 可以想成一個「翻譯機兼搬運工」: 以前用 JDBC 存資料,你要自己寫一堆 SQL、自己把 `ResultSet` 一格一格轉成 Java 物件,很累。Hibernate 幫你把這些髒活都包了,你只要把 Java 物件交給它,它就會自動翻譯成資料庫看得懂的 SQL,存進對應的資料表。

這種「Java 類別 ↔ 資料表」的對應規則,術語叫 **ORM (Object-to-Relational Mapping)**。因為 Java 習慣用 camelCase(如 `firstName`),資料庫欄位習慣用 snake_case(如 `first_name`),所以需要一套映射規則把兩邊兜起來,可以用 XML 設定,但更常見的是用 Java 註解。

JPA (Jakarta Persistence API) 則是這件事的「國家標準規格」——它本身只定義了一組介面,並不是真正做事的程式。Hibernate 就是把這套規格「做出來」的其中一個實作(另一個常見的是 `EclipseLink`),也是 Spring Boot 的預設實作。這樣設計的好處是「不被單一廠商綁死」: 程式碼只針對 JPA 介面寫,理論上哪天要換成別的實作,只需要改設定,不用重寫商業邏輯。

底層關係是一層包一層的抽象:

```
Your Java App → JPA/Hibernate → JDBC → Database
```

也就是說,即使你用的是 JPA 的高階寫法,Hibernate 骨子裡最後還是透過 JDBC 跟資料庫溝通,只是這些細節都被藏起來了。

存物件跟讀物件的基本寫法長這樣,核心工具是 `EntityManager`:

```
// 存物件
Student theStudent = new Student("Paul", "Doe", "paul@luv2code.com");
entityManager.persist(theStudent);

// 用主鍵讀物件
Student myStudent = entityManager.find(Student.class, theId);

// 查一批物件(用 JPQL, 不是 SQL)
TypedQuery<Student> theQuery = entityManager.createQuery("from Student",
```

```
Student.class);  
List<Student> students = theQuery.getResultList();
```

JPA/Hibernate 要做的核心任務,說穿了就是完整的 **CRUD**:Create(建立)、Read(讀取)、Update(更新)、Delete(刪除)。

本節後半段是 MySQL 環境的安裝與初始化(安裝 MySQL Server 與 Workbench、建立 `springstudent` 使用者、跑 `01-create-user.sql` 與 `02-student-tracker.sql` 建出 `student_tracker` 資料庫與 `student` 資料表,欄位為 `id` / `first_name` / `last_name` / `email`),屬於一次性的環境設定,重點記住:之後所有 Java 端的實體欄位都要對齊這張表的欄位名稱。

19. Automatic Data Source Configuration

Spring Boot 最貼心的地方之一,就是「你只要放對依賴,它就自動幫你接好電線」。只要 `pom.xml` 裡有 `mysql-connector-j` (JDBC 驅動)跟 `spring-boot-starter-data-jpa` (ORM 支援),Spring Boot 啟動時就會自動建立好 `DataSource` 與 `EntityManager` 這兩個 Bean,不需要自己寫任何設定類別,連 JDBC driver class 名稱都不用手動指定——Spring Boot 會從 `spring.datasource.url` 自動判斷。你只要在 `application.properties` 填連線資訊:

```
spring.datasource.url=jdbc:mysql://localhost:3306/student_tracker  
spring.datasource.username=springstudent  
spring.datasource.password=springstudent
```

要驗證這個機制是不是真的有生效,一個很實用的土法煉鋼技巧是「故意打錯密碼」:改成錯的密碼重啟,應用程式會在啟動階段就噴錯(`Access denied for user ... using password: YES`),證明它真的有嘗試連線;改回正確密碼後,日誌會出現 `Added connection ...` 與 `Started ... successfully`,才代表連線真的建立起來了。

為了先專注在 Hibernate/JPA 的程式邏輯,這裡用最陽春的命令列程式來練習,核心工具是 `CommandLineRunner`——可以把它想成一個「開機自動啟動的鉤子」,規則是:等所有 Spring Bean 都載入完成之後,才會執行裡面的程式碼:

```
@Bean  
public CommandLineRunner commandLineRunner(String[] args) {  
    return runner -> {  
        System.out.println("Hello World");  
    };  
}
```

開發階段常嫌 Spring Boot 啟動時印出的一大堆訊息很吵,可以透過兩個設定讓畫面乾淨一點(注意:只建議開發用,正式環境通常要保留 INFO 甚至更詳細的日誌以利監控):

```
spring.main.banner-mode=off  
logging.level.root=warn
```

降低日誌層級不代表出事了看不到——**WARN**、**ERROR** 一律還是會顯示,只是把大量 **INFO** 等級的背景雜訊(如連線池初始化過程)隱藏掉而已。

20. 儲存 Java 物件 (Save a Java Object)

這節開始搭建真正的 CRUD 架構,核心是 **DAO(Data Access Object)設計模式**——可以想成餐廳的「內外場分工」:應用程式(外場)不用知道資料庫怎麼存取的細節,所有跟資料庫打交道的事都交給 DAO(後場廚房)處理,兩邊透過介面溝通,職責分得很乾淨。

DAO 內部真正做事的是 **EntityManager**,它是 JPA 裡最核心的物件,所有實體的存取都要透過它。這裡會出現一個常見疑問:市面上很多教學直接用更方便的 **JpaRepository**,為什麼這裡要從更底層的 **EntityManager** 學起?簡單比較:

特性	EntityManager	JpaRepository
核心價值	低階控制、靈活(可寫原生 SQL、預存程序等客製查詢)	開箱即用的標準 CRUD,內建分頁與排序
開發效率	需要自己寫較多程式碼	大幅減少樣板程式碼

建議是兩者都要會——只會 **JpaRepository** 的人,遇到複雜問題容易卡住,因為不了解背後在幹嘛;先摸熟 **EntityManager** 的底層邏輯,再學 **JpaRepository** 加速開發,是比較紮實的學習路線。

在寫 DAO 之前,要先把 **Student** 類別變成 JPA 實體(**Entity**)。規則很簡單:類別上要有 **@Entity** 跟 **@Table(name="student")**,主鍵欄位要有 **@Id**,並用 **@GeneratedValue(strategy = GenerationType.IDENTITY)** 告訴 JPA「這個 ID 交給資料庫的 AUTO_INCREMENT 自動生成就好,你不用管」,再用 **@Column(name="...")** 把 Java 屬性名對應到資料庫欄位名(這個註解雖然可省略、預設會用同名欄位,但強烈建議明寫,不然日後改欄位名稱容易跟資料庫對不上,造成破壞性變更):

```
@Entity
@Table(name="student")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name="id")
    private int id;

    @Column(name="first_name")
    private String firstName;

    @Column(name="last_name")
    private String lastName;

    @Column(name="email")
    private String email;

    // 無參數建構子(JPA 硬性要求)、全參數建構子、getter/setter、toString
}
```

有個容易忽略的陷阱:JPA 規定實體類別「必須有無參數建構子」。Java 原本會自動幫沒寫建構子的類別補一個無參數建構子,但只要你手動寫了一個帶參數的建構子,這個自動補的機制就會消失,所以要自己再明確寫一個無參數建構子出來。

主鍵生成策略除了最常用的 `IDENTITY`,還有 `AUTO`(讓 JPA 自己挑)、`SEQUENCE`、`TABLE`、`UUID` 等選項,一般團隊優先用 `IDENTITY` 就夠了。

DAO 的實作模式固定是三步驟:定義介面 → 寫實作類別(建構子注入 `EntityManager`,存檔動作要加 `@Transactional`) → 在主程式呼叫測試:

```
public interface StudentDAO {
    void save(Student theStudent);
}

@Repository
public class StudentDAOImpl implements StudentDAO {

    private EntityManager entityManager;

    @Autowired
    public StudentDAOImpl(EntityManager theEntityManager) {
        entityManager = theEntityManager;
    }

    @Override
    @Transactional
    public void save(Student theStudent) {
        entityManager.persist(theStudent);
    }
}
```

`@Repository` 是 `@Component` 的子註解,除了能被組件掃描自動註冊成 Bean,還會自動把底層的 JDBC 例外轉換成 Spring 統一的資料存取例外,省去一堆 try-catch。`@Transactional` 則負責在背景自動開啟/結束交易——凡是「會修改資料庫」的操作(存、改、刪)都要加,單純查詢(唯讀)則不需要。

另外提一個實用小技巧:如果想讓自動遞增的 ID 不要從 1 開始,可以用 `ALTER TABLE student_tracker.student AUTO_INCREMENT = 3000;`;想把計數器重設回 1,則用 `TRUNCATE student_tracker.student;`(注意這會清空整張表)。

21. 使用 JPA 讀取物件

讀取單一物件最基本的方式就是靠主鍵查,`entityManager.find()` 吃兩個參數:第一個是實體類別(如 `Student.class`),第二個是主鍵值,找不到的話會回傳 `null`(不會丟例外)。

```
Student myStudent = entityManager.find(Student.class, 1);
```

DAO 開發流程一樣照三步驟走:介面加方法 → 實作 → 主程式呼叫測試。這裡有個重點跟 `save` 不同——`findById` 這種純查詢方法不需要加 `@Transactional`,因為它不會動到資料庫內容,是唯讀操作:

```
@Override
public Student findById(Integer id) {
    return entityManager.find(Student.class, id);
}
```

完整的測試流程通常是「建立 → 存檔 → 拿到自動生成的 ID → 用這個 ID 查回來 → 印出來確認」,前後串成一個閉環,確保存進去的資料真的能原樣讀出來,這也是驗證整個 JPA 映射有沒有寫對最直接的方法。

22. 查詢物件 (Query Objects)

要一次查多筆資料,要用 JPA 自己的查詢語言 **JPQL**。JPQL 長得很像 SQL,也支援 **WHERE**、**LIKE**、**ORDER BY**、**JOIN**、**IN** 這些熟悉的關鍵字,但有一個關鍵差異一定要記住:**SQL 操作的是「資料表與欄位」**,**JPQL 操作的是「Entity 名稱與 Entity 欄位」**。也就是說,JPQL 裡寫的是 Java 類別名稱與屬性名(如 **Student**、**lastName**),不是資料庫裡的表名跟欄位名(如 **student**、**last_name**)——這是最容易搞混、也最容易寫錯的地方。

```
// 查所有 Student
TypedQuery<Student> theQuery = entityManager.createQuery("FROM Student",
Student.class);
List<Student> students = theQuery.getResultList();

// 條件查詢 + LIKE 模糊比對(% 是萬用字元)
TypedQuery<Student> theQuery2 = entityManager.createQuery(
    "FROM Student WHERE email LIKE '%luv2code.com'", Student.class);
```

實務上很少會把查詢條件寫死在字串裡(硬編碼),因為條件通常來自使用者輸入。這時候要用具名參數,寫法是在 JPQL 裡用冒號 : 開頭定義佔位符,再用 **setParameter()** 綁定實際值:

```
public List<Student> findByLastName(String theLastName) {
    TypedQuery<Student> theQuery = entityManager.createQuery(
        "FROM Student WHERE lastName=:theData", Student.class);
    theQuery.setParameter("theData", theLastName);
    return theQuery.getResultList();
}
```

有個小細節值得注意:上面範例都省略了 **SELECT**,直接寫 **FROM Student** 也能跑,這其實是 Hibernate 比較「寬容」的 HQL(Hibernate Query Language)寫法。如果要嚴格符合 JPQL 標準,規範上是要寫完整的 **SELECT ... FROM ... WHERE ...**,並且要用一個別名(identification variable)來代表這個實體,例如 **s**,之後要引用欄位就要寫成 **s.email** 這種「別名.欄位名」的形式:

```
TypedQuery<Student> theQuery = entityManager.createQuery(
    "select s FROM Student s WHERE s.email LIKE '%luv2code.com'",
    Student.class);
```

`findAll()` 的實作邏輯完全一樣的套路:建查詢 → 執行 `getResultList()`:

```
@Override
public List<Student> findAll() {
    TypedQuery<Student> theQuery = entityManager.createQuery("FROM
Student", Student.class);
    return theQuery.getResultList();
}
```

23. 實作按姓氏排序 (Sort by Last Name)

排序功能就是在 JPQL 尾巴加一個 `ORDER BY` 子句,一樣要注意:排序欄位要用 **Entity** 的欄位名稱(如 `lastName`),不是資料庫欄位名(`last_name`)。不特別指定的話預設是升冪(A 到 Z),要倒過來就加 `DESC`,想明確表達升冪也可以寫 `ASC`(雖是預設值,但寫出來可讀性更好):

```
public List<Student> findAll() {
    TypedQuery<Student> theQuery = entityManager.createQuery(
        "FROM Student order by lastName desc", Student.class);
    return theQuery.getResultList();
}
```

用 `desc` 測試時,結果會從 Z 排到 A(例如 `Public → Duck → Doe → Applebaum`);拿掉 `desc/asc` 字樣,就會自動退回預設的升冪排序。這一節的重點其實很單純:JPQL 的排序語法跟 SQL 幾乎一樣好記,唯一要小心的坑仍然是「欄位名稱要用 Java 屬性名,不是資料庫欄位名」這個貫穿整個 JPQL 的鐵律。

24. 根據 Java 程式碼生成資料表 (Creating Tables based on Java Code)

前面都是先手動在 MySQL Workbench 建好表,Java 這邊再對齊。Hibernate 其實還有一招「反過來做」的能力:只要 Entity 類別上的註解寫好(`@Table`、`@Column` 等),Hibernate 可以在應用程式啟動時自動把對應的 `CREATE TABLE` SQL 生出來並執行,開發者完全不用手動寫建表腳本:

```
@Entity
@Table(name="student")
public class Student {
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    @Column(name="id")
    private int id;
    // ...
}
```

```
create table student (id integer not null auto_increment, email
varchar(255),
    first_name varchar(255), last_name varchar(255), primary key (id))
```

控制這個行為的開關是 `spring.jpa.hibernate.ddl-auto`,這是全節最重要也最容易踩雷的設定,務必分清每個值的行為:

設定值	行為	適用場景
<code>none</code>	什麼都不做	正式環境
<code>validate</code>	只檢查 Entity 跟資料庫結構是否一致,不做任何修改	正式環境
<code>update</code>	依 Entity 變動自動 ALTER 資料表,不會刪表、資料會保留	個人小專案(仍有風險)
<code>create</code>	啟動時先刪表再重建,所有既有資料全部消失	開發/測試階段
<code>create-drop</code>	跟 <code>create</code> 一樣先刪再建,但應用程式關閉時還會再刪一次表	單元測試

一句話記住:`create` 每次啟動都會把表砍掉重練,`update` 則是保留資料、只做結構上的增量調整,但 `update` 一樣有風險——它會自動 ALTER 資料庫結構,在多團隊共用同一個資料庫的情境下很危險。

絕對不要在正式環境的資料庫上使用 `create` 或 `update`!`create` 會讓所有正式資料瞬間消失,`update` 則可能悄悄改動別的團隊也在用的資料表結構。正式環境的資料庫結構變更,應該交給 DBA 用受控的 SQL 腳本執行,腳本可以走版本控制、code review,也可以搭配 Liquibase、Flyway 這類 schema 遷移工具管理演進歷史——這樣的可審查性是自動生成機制完全沒有的。

自動生成比較適合的場景是:資料庫整合測試(搭配記憶體內資料庫)、或個人的小型 side project。

開發時如果想親眼看到 Hibernate 到底送出了什麼 SQL、綁了什麼參數,可以開這兩行 log(注意這也只建議開發用):

```
logging.level.org.hibernate.SQL=debug
logging.level.org.hibernate.orm.jdbc.bind=trace
```

`debug` 會印出完整的 SQL 語句(如 `insert into student (...) values (?, ?, ?)`),`trace` 則會把綁進問號裡的實際數值也印出來,對除錯或單純想搞懂 Hibernate 幕後在做什麼非常有幫助。文中也做了一個破壞性測試示範:手動把 `student` 表 `DROP TABLE` 掉之後重啟程式,程式會直接噴出 `Table 'student_tracker.student' doesn't exist` 的錯誤——這也側面說明了 `ddl-auto=none`(什麼都不做)在正式環境才是安全的預設值,一旦表結構跟程式碼對不上,系統就該讓你知道,而不是自作主張幫你改。

25. Spring REST 開發實作案例

REST API 說穿了就是「餐廳的菜單跟點餐窗口」:前端(客戶端)不用知道廚房(伺服器)怎麼煮菜、用什麼食材,只要照著菜單格式點餐,廚房就會把做好的菜(資料)端出來。REST 全名是 REpresentational State Transfer,重點特性有兩個:

- **語言獨立(Language Independent)**:點餐的人可以講中文、英文、日文(Java、C#、Swift、Python...都行),廚房只要聽得懂 HTTP 協定這套「通用語」就能回應,所以同一個後端服務可以同時給 Web、手機 App、桌面程式共用。
- **資料格式自由**:最常用的是 JSON,輕量、純文字、幾乎所有語言都能解析,是目前業界的主流選擇(XML 也可以,但比較少用)。

課程用天氣 App、貨幣轉換 App、電影訂票 App、CRM(客戶關係管理)App 這幾個例子說明同一種模式:客戶端丟一個查詢請求(城市名、轉換金額、電影地點...),外部服務回傳對應的 JSON 資料,客戶端自己決定要怎麼呈現。這也是這門課接下來要做的「CRM Service」的雛形——伺服器只負責吐資料,不管前端長什麼樣子,這就是「資料與呈現分離」的精神。順帶一提,REST API、RESTful API、REST Web Service 這些說法其實都是同一件事,不用糾結用詞。

JSON 基礎語法(之後每一節都會用到,先搞熟):

```
{
  "id": 14,
  "firstName": "Mario",
  "lastName": "Rossi",
  "active": true,
  "courses": null,
  "languages": ["Java", "C#", "Python"],
  "address": {
    "city": "Philadelphia",
    "state": "Pennsylvania"
  }
}
```

- 物件用 `{ }`,裡面是一組一組的「名稱: 值」,名稱一定要雙引號包起來。
- 值可以是數字(不用引號)、字串(要引號)、布林值 `true/false`、`null`(代表沒有值)、陣列 `[ ]`,甚至是另一個巢狀物件——想嵌多深都可以。

最後,REST 常常搭配 HTTP 方法對應到資料庫的 CRUD 操作,這個對照表之後會一直用到:

HTTP Method	CRUD 操作
POST	Create(新增)
GET	Read(讀取單筆或列表)
PUT	Update(更新)
DELETE	Delete(刪除)

26. HTTP 訊息結構

不管是「請求」還是「回應」,HTTP 訊息都拆成三塊,可以想成寄一個包裹:收件地址與動作(request line)、包裹上貼的標籤資訊(headers)、箱子裡真正的東西(body)。

請求(Request)三部分:

- **Request line:**HTTP 方法,例如 `GET`、`POST`、`DELETE`。
- **Header variables:**這次請求的附加資訊(metadata)。
- **Message body:**實際要送出去的內容,例如新增資料時帶的那筆資料。

回應(Response)三部分:

- **Response line:**協定版本 + 狀態碼(status code),例如 `200`、`404`、`500`。

- **Header variables**: 回應資料的說明, 例如內容類型、資料大小。
- **Message body**: 伺服器真正回傳的內容(通常是 JSON 或 XML)。

狀態碼分五個區間, 大概記住這幾類就夠用:

範圍	意義
100-199	資訊性
200-299	成功
300-399	重新導向
400-499	客戶端錯誤(例如 404 找不到資源)
500-599	伺服器錯誤

MIME Content Type 是用來標明 body 裡裝的到底是什麼格式, 語法是 `type/subtype`, 像 `application/json` 就是告訴客戶端「這包東西是 JSON, 請用處理 JSON 的方式打開它」, 常見的還有 `text/html`、`application/xml`。

測試 REST API 最常用兩個工具: 瀏覽器(只能發 GET 請求, 簡單看看資料)跟 **Postman**(GUI 工具, 能設定各種 HTTP 方法、Headers、Body, 是實務上做 API 測試的標配)。用 Postman 測試的基本流程是: 選 HTTP Method → 貼上 URL → 按 Send → 在下方 Response 區看狀態碼、回應時間跟 Body 內容, 同時也要留意 Headers 分頁裡的 **Content-Type** 等元資料。課程用免費的 jsonplaceholder.typicode.com 當練習用的假 API 來回。

27. Spring REST Controller 開發流程

寫第一個 REST 服務, 流程其實很固定, 兩個步驟走完就有一個能動的 API:

步驟一: 加 Maven 依賴。如果是手動改 `pom.xml`, 加入 Spring Boot Starter Web(內建 Tomcat 跟 RESTful 支援); 但更方便的做法是直接在 Spring Initializr(start.spring.io) 建立專案時, 勾選 "Web" 依賴, `pom.xml` 就自動幫你設好, 連版本都不用煩惱——記得選最新的**正式發佈版**, 不要選帶 **SNAPSHOT** 字樣的(那是還在開發中、不穩定的版本)。

步驟二: 寫 REST Controller。用三個註解就能把一個普通類別變成 REST 服務入口:

```
@RestController
@RequestMapping("/test")
public class DemoRestController {

    @GetMapping("/hello")
    public String sayHello() {
        return "Hello World!";
    }
}
```

- **@RestController**: 告訴 Spring 這個類別是專門處理 REST 請求的控制器(跟傳統回傳 HTML 頁面的 **@Controller** 不同, 它會直接把回傳值轉成資料格式回應)。
- **@RequestMapping("/test")**: 設定這個類別底下所有端點的「共同前綴路徑」。

- `@GetMapping("/hello")`:把 GET 請求 `/test/hello` 對應到 `sayHello()` 這個方法。

跑起來後,不管用瀏覽器輸入 `http://localhost:8080/test/hello`,還是用 Postman 發 GET 請求到同一個網址,都能拿到 **Hello World!**,狀態碼 200 OK。實務上專案會把 REST Controller 獨立放在一個 package(例如 `com.luv2code.demo.rest`)裡,跟其他業務邏輯分開管理,方便日後維護。

28. Java JSON 資料繫結 (Data Binding)

資料繫結(Data Binding)講白話就是「JSON 跟 Java 物件互相翻譯」:JSON 進來變成 Java POJO,Java POJO 出去變成 JSON。這個翻譯的工作在 Spring 裡是交給一個叫 **Jackson** 的函式庫在背後默默處理,只要專案有加 Spring Boot Starter Web,Jackson 就自動包在裡面,完全不用手動設定。你可能會聽到不同說法(Mapping、Serialization/Deserialization、Marshalling/Unmarshalling),但講的都是同一件事。

Jackson 的翻譯規則,靠的是 getter/setter,不是直接動私有欄位:

- JSON → POJO:Jackson 看到 `"firstName": "Mario"`,就會去呼叫 POJO 的 `setFirstName("Mario")`。
- POJO → JSON:Jackson 呼叫 `getFirstName()` 取值,組成 `"firstName": "Mario"` 這個鍵值對。

推導規則很機械:取 JSON 的鍵名、首字大寫、前面加上 `set`,例如 `firstName` → `setFirstName`。這代表如果你的 **POJO** 少了對應的 **getter/setter**,資料繫結就會失敗,這是最容易踩的坑。

實作 **Student** REST 服務的完整流程:

```
// Student.java (POJO)
public class Student {
    private String firstName;
    private String lastName;

    public Student() {}
    public Student(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }
    // getters / setters ...
}

// StudentRestController.java
@RestController
@RequestMapping("/api")
public class StudentRestController {

    @GetMapping("/students")
    public List<Student> getStudents() {
        List<Student> theStudents = new ArrayList<>();
        theStudents.add(new Student("Poornima", "Patel"));
        theStudents.add(new Student("Mario", "Rossi"));
        theStudents.add(new Student("Mary", "Smith"));
        return theStudents;
    }
}
```

方法回傳 `List<Student>`, Jackson 就會自動把整個列表轉成 JSON 陣列送給客戶端, Controller 完全不用手動拼 JSON 字串。

小重構重點:上面這樣寫,每次有請求進來都要重新 new 一次 `ArrayList`、重新塞資料,很浪費。改善做法是把 `theStudents` 提升成類別欄位,搭配 `@PostConstruct` 註解——這個註解標記的方法只會在 Bean 建構完成後執行一次,很適合拿來做初始化(之後接資料庫也是類似邏輯,只是換成查詢資料庫):

```
private List<Student> theStudents;

@PostConstruct
public void loadData() {
    theStudents = new ArrayList<>();
    theStudents.add(new Student("Poornima", "Patel"));
    theStudents.add(new Student("Mario", "Rossi"));
    theStudents.add(new Student("Mary", "Smith"));
}

@GetMapping("/students")
public List<Student> getStudents() {
    return theStudents;
}
```

29. Path Variables (路徑變數)

路徑變數就是 URL 裡用大括號 `{}` 包起來的佔位符,用來讓同一個端點根據不同的值查詢不同的資源——就像網購時網址裡的訂單編號,同一套查詢邏輯,換個編號就查到不同筆訂單。例如 `/api/students/{studentId}`, 呼叫 `/api/students/1` 就是要第 1 筆學生資料。

實作上用 `@PathVariable` 註解把路徑變數綁定到方法參數:

```
@GetMapping("/students/{studentId}")
public Student getStudent(@PathVariable int studentId) {
    return theStudents.get(studentId);
}
```

要注意的地方:預設情況下,`{}` 裡的名稱(`studentId`)必須跟方法參數的名稱(`int studentId`)完全一致, Spring 才能自動完成綁定,這是新手很容易忽略、對不上就綁定失敗的細節。

目前這個範例還沒接資料庫,是暫時拿 `studentId` 當作 `ArrayList` 的索引值來偷懶查資料,只是為了先讓流程跑起來,之後會換成真正的資料庫查詢。

這裡埋了一個大坑:如果傳入的 ID 超出範圍(例如 `/api/students/9999`,但列表只有 3 筆),`theStudents.get(studentId)` 會丟出 `IndexOutOfBoundsException`,伺服器直接回傳一個很不友善的 `500 Internal Server Error`——這種原始錯誤畫面對客戶端來說完全看不懂發生什麼事。這就是下一節「例外處理」要解決的問題:把這種醜陋的錯誤,轉換成結構清楚、客戶端看得懂的 JSON 錯誤訊息。

30. Spring REST 例外處理目標

當使用者查不存在的資料(比如上一節那個 ID 9999 的例子)時,理想的 API 不該回傳一坨看不懂的錯誤堆疊,而應該回傳一個結構化、看得懂的 JSON,而且用正確的狀態碼(404 而不是籠統的 500)。目標長這樣:

```
{
  "status": 404,
  "message": "Student id not found - 9999",
  "timeStamp": 15261496
}
```

要做到這件事,標準流程分四步:

第一步:自訂錯誤回應類別——就是一個普通 POJO,定義要回給客戶端的 JSON 結構長什麼樣:

```
public class StudentErrorResponse {
    private int status;
    private String message;
    private long timeStamp;
    // constructors, getters/setters ...
}
```

第二步:自訂例外類別——繼承 `RuntimeException`(unchecked exception,呼叫端不用強制寫 try-catch 或 throws,程式碼比較乾淨):

```
public class StudentNotFoundException extends RuntimeException {
    public StudentNotFoundException(String message) {
        super(message);
    }
}
```

第三步:在 `Service` 裡主動丟出例外——查詢前先檢查 ID 有效性,無效就丟出剛剛定義的例外(換成資料庫查詢邏輯,就是「查無結果就丟例外」的概念):

```
@GetMapping("/students/{studentId}")
public Student getStudent(@PathVariable int studentId) {
    if (studentId >= theStudents.size() || studentId < 0) {
        throw new StudentNotFoundException("Student id not found - " +
studentId);
    }
    return theStudents.get(studentId);
}
```

第四步:用 `@ExceptionHandler` 接住例外,轉成 `ResponseEntity`——`ResponseEntity` 是 HTTP 回應的包裝器,可以精確控制狀態碼、Headers、Body 三樣東西:

```
@ExceptionHandler(StudentNotFoundException.class)
public ResponseEntity<StudentErrorResponse>
handleException(StudentNotFoundException exc) {
    StudentErrorResponse error = new StudentErrorResponse();
    error.setStatus(HttpStatus.NOT_FOUND.value());
    error.setMessage(exc.getMessage());
    error.setTimestamp(System.currentTimeMillis());
    return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
}
```

容易忽略的陷阱:上面這個 handler 只接得住 `StudentNotFoundException`。如果使用者輸入的路徑變數根本不是數字(例如 `/api/students/abcde`),Spring 在把字串轉成 `int` 這一步就會失敗,丟出 `MethodArgumentTypeMismatchException`,而這個例外不會被你自訂的 handler 接住,客戶端只會收到 Spring Boot 預設醜醜的 400 錯誤格式,不是你想要的 JSON。解法是再加一個「全捕獲(catch-all)」handler,參數用最上層的 `Exception`,當作最後一道防線,統一回傳 400:

```
@ExceptionHandler(Exception.class)
public ResponseEntity<StudentErrorResponse> handleException(Exception exc)
{
    StudentErrorResponse error = new StudentErrorResponse();
    error.setStatus(HttpStatus.BAD_REQUEST.value());
}
```

```

    error.setMessage(exc.getMessage());
    error.setTimestamp(System.currentTimeMillis());
    return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
}

```

再進一步:全域例外處理(**@ControllerAdvice**)。如果專案有很多個 Controller,每個都複製貼上同一套 **@ExceptionHandler** 邏輯,會很難維護。**@ControllerAdvice** 的作用類似攔截器(Interceptor)/過濾器(Filter),可以把例外處理邏輯集中放到一個獨立類別,讓所有 Controller 共用,是 AOP(面向切面程式設計)把「例外處理」這種橫切關注點抽離出來的典型案例:

```

@ControllerAdvice
public class StudentRestExceptionHandler {

    @ExceptionHandler(StudentNotFoundException.class)
    public ResponseEntity<StudentErrorResponse>
    handleException(StudentNotFoundException exc) {
        StudentErrorResponse error = new StudentErrorResponse();
        error.setStatus(HttpStatus.NOT_FOUND.value());
        error.setMessage(exc.getMessage());
        error.setTimestamp(System.currentTimeMillis());
        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<StudentErrorResponse> handleException(Exception
    exc) {
        StudentErrorResponse error = new StudentErrorResponse();
        error.setStatus(HttpStatus.BAD_REQUEST.value());
        error.setMessage(exc.getMessage());
        error.setTimestamp(System.currentTimeMillis());
        return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
    }
}

```

重構時只是把 **@ExceptionHandler** 方法整包搬到這個新類別,邏輯完全不用改。這樣做的好處是邏輯解耦(Controller 只管業務邏輯,錯誤處理交給 Advice 層)、可重用(多個 Service 共用同一套錯誤處理),也是業界公認處理 REST 錯誤的標準做法。

31. Spring REST API 設計

設計 REST API 之前,先問自己兩個問題:誰會用這個 API?他們會怎麼用?想清楚需求後,設計流程可以拆成三步:

1. 審查需求 → 2. 找出主要「資源」(名詞) → 3. 用 HTTP 方法定義動作(動詞)

用員工目錄系統當例子,需求就是標準 CRUD:列出員工、依 ID 查一個、新增、更新、刪除。這裡的「資源」就是 **employee**,而且慣例上端點路徑要用複數形式,例如 **/api/employees**,而不是 **/api/employee**。

最重要的觀念是:路徑只能是名詞,動作要靠 HTTP 方法來表達。可以把 REST API 想成一間圖書館的借還書櫃檯——櫃檯本身(路徑)永遠叫「書籍」,你要做的事情(借、還、查、改)是跟櫃員說的動作(HTTP 方法),而不是把

動作寫進櫃檯招牌裡。所以像 `/api/addEmployee`、`/api/deleteEmployee` 這種把動詞塞進路徑的寫法,是典型的「反模式」(anti-pattern),業界公認不好。正確做法是:

HTTP Method	Endpoint	CRUD Action
POST	<code>/api/employees</code>	新增員工
GET	<code>/api/employees</code>	讀取員工列表
GET	<code>/api/employees/{employeeId}</code>	讀取單一員工
PUT	<code>/api/employees/{employeeId}</code>	更新員工
DELETE	<code>/api/employees/{employeeId}</code>	刪除員工

需要指定特定資源時,用路徑變數(Path Parameter)如 `{employeeId}`;而 POST、PUT 這種要傳資料的方法,資料會包在 Request Body 裡以 JSON 格式傳送。

這套設計原則不是憑空想像,現實世界的大公司 API 幾乎都照這個邏輯走:PayPal 發票 API、GitHub Repositories API(路徑變數用冒號 `:owner/:repo`,跟 `{owner}/{repo}` 意思相同)都是如此。連 Salesforce 這種大廠,雖然在某些端點用了單數形式 `/individual` 而非複數,細節上有出入,但核心邏輯——用 HTTP 方法定義動作——仍然一致。也就是說,複數命名是「建議慣例」,不是鐵律,重點是不要把動詞塞進路徑。

32. Spring Boot REST API 即時專案

這節開始動手做一個完整的員工目錄專案,目標是撈起這樣的分層架構:

Employee REST Controller ↔ Employee Service ↔ Employee DAO ↔ 資料庫

三層各司其職:Controller 接 HTTP 請求、Service 放業務邏輯、DAO 負責跟資料庫打交道。整個開發流程照順序來:設定資料庫 → 用 Spring Initializr 建專案 → 依序做出「查全部」「查單一」「新增」「更新」「刪除」五支 API。

資料庫準備:執行 `employee.sql` 腳本會自動建立 `employee_directory` 資料庫與 `employee` 資料表(欄位 `id`、`first_name`、`last_name`、`email`),並塞入五筆範例資料方便測試。

建立專案:用 start.spring.io 選 Maven + Java,務必勾四個依賴:Spring Web(處理 HTTP)、Spring Data JPA(資料庫存取)、Spring Boot DevTools(改程式碼自動重載,不用手動重啟)、MySQL Driver(連 MySQL)。

Entity 類別:`Employee` 類別要用 `@Entity` 標記為 JPA 管理的實體,`@Table(name="employee")` 對應到資料表。每個欄位要用 `@Column(name="...")` 明確指定對應的資料庫欄位名——這裡有個常見地雷:Java 習慣用 camelCase(如 `firstName`),但資料庫欄位通常是 snake_case(`first_name`),兩者名字對不上會出錯,務必仔細核對。主鍵欄位要加 `@Id` 加 `@GeneratedValue(strategy = GenerationType.IDENTITY)`,讓資料庫自動遞增產生 ID。另外 JPA 規範要求實體類別必須有一個無參數建構子(給框架反射用),手動建立物件用的全參數建構子則要排除 `id`,因為 `id` 是資料庫自動生成的,不該由開發者傳入。

DAO 層(先用原生 JPA API 寫):先定義 `EmployeeDAO` 介面,再寫 `EmployeeDAOJpaImpl` 實作類別,標上 `@Repository` 註解(告訴 Spring 這是資料存取層元件,也讓 Spring 幫忙轉譯資料庫例外)。核心做法是透過建構子注入 `EntityManager`(Spring Boot 自動幫你建好,你只要接手用):

```
@Repository
public class EmployeeDAOJpaImpl implements EmployeeDAO {
    private EntityManager entityManager;

    @Autowired
    public EmployeeDAOJpaImpl(EntityManager theEntityManager) {
        entityManager = theEntityManager;
    }

    @Override
    public List<Employee> findAll() {
        TypedQuery<Employee> theQuery =
            entityManager.createQuery("from Employee", Employee.class);
        return theQuery.getResultList();
    }
}
```

`findAll()` 用 JPQL(用 Java 類別名 `Employee` 而非表名去查)建立查詢再執行,取得結果列表。

Controller 層:一開始用「快速粗暴」(quick and dirty)的方式,直接把 `EmployeeDAO` 注入 Controller 求快,用 `@RestController` + `@RequestMapping("/api")` 標記類別,再用 `@GetMapping("/employees")` 暴露端點,方法內容就是直接委派給 DAO。啟動應用後訪問 `http://localhost:8080/api/employees` 就能看到 JSON,同時用 MySQL Workbench 查資料庫比對,確認資料一致。不過這種直接把 DAO 塞進 Controller 的做法只是暫時的,下一節就會重構成正式的三層架構。

33. 引入 Service Layer

上一節 Controller 直接呼叫 DAO 只是權宜之計,這節把 Service 層加進來,變成 Controller ↔ Service ↔ DAO 的正式三層架構。可以把 Service 層想成餐廳裡的「內場經理」:客人(Controller)不會直接衝進廚房跟每個廚師(各個 DAO)喊菜,而是跟內場經理下單,經理再去協調廚房各站(可能同時整合 Employee DAO、Skills DAO、Payroll DAO 等多個資料來源),最後端出一份整合好的結果給客人。這種設計叫 Service Facade 模式。

在簡單系統裡,Service 常常只是單純「委派」(delegate)角色——接到請求就直接轉呼叫對應的 DAO,不做複雜邏輯。即便如此,建立這層架構仍是業界最佳實踐,好處是系統擴充時很容易加新邏輯進去,大型企業級應用普遍採用。

Spring 提供專用的 `@Service` 註解,它是 `@Component` 的特化版本,跟 `@RestController`、`@Repository` 屬於同一家族,都會被組件掃描自動註冊進容器。

實作步驟:先定義 `EmployeeService` 介面(方法基本上跟 `EmployeeDAO` 對應),再寫 `EmployeeServiceImpl` 實作,透過建構子注入 `EmployeeDAO`:

```
@Service
public class EmployeeServiceImpl implements EmployeeService {
    private EmployeeDAO employeeDAO;

    public EmployeeServiceImpl(EmployeeDAO theEmployeeDAO) {
        this.employeeDAO = theEmployeeDAO;
    }
}
```

```
@Override
public List<Employee> findAll() {
    return employeeDAO.findAll();
}
```

接著重構 Controller,把原本注入 `EmployeeDAO` 的地方全部改成注入 `EmployeeService`,方法內部呼叫也從 `employeeDAO.findAll()` 換成 `employeeService.findAll()`。這樣 Controller 就完全不需要知道 DAO 或資料庫的存在,職責更單一。

事務管理(Transactional)的關鍵規則:凡是會修改資料庫的操作(新增、更新、刪除),都要在 **Service** 層加 `@Transactional`,而不是放在 DAO 層。原因是萬一個 Service 方法內要呼叫多個 DAO 做事,整個流程必須被視為一個不可分割的原子操作——這正是 Service 層存在的意義之一,DAO 層只管單一資料存取,不該管交易邊界。

```
@Transactional
@Override
public Employee save(Employee theEmployee) {
    return employeeDAO.save(theEmployee);
}
```

DAO 層這節也補齊了完整 CRUD:`findById` 用 `entityManager.find(Employee.class, theId)`;`save` 用 `entityManager.merge()` 同時處理新增與更新——重點是一定要回傳 `merge()` 產生的物件,而不是原本傳入的物件,因為新增時資料庫會產生新的自動遞增 ID,只有 `merge()` 回傳的物件才帶有這個新 ID,回傳原始物件的話後續邏輯會拿不到正確 ID;`deleteById` 則是先 `find` 找到實體,再用 `entityManager.remove()` 刪除。這三個方法統統不該加 `@Transactional`,理由同上,一律交給 Service 層統一管理。

34. 建立「新增員工」的 API

這節實作完整的 CRUD 端點:新增(POST)、更新(PUT)、部分更新(PATCH)、刪除(DELETE)。

新增員工(POST):用 `@PostMapping("/employees")` 搭配 `@RequestBody` 把請求主體的 JSON 自動綁定成 `Employee` 物件。有個容易忽略的細節:如果客戶端不小心在 JSON 裡帶了 `id`,必須手動把它清掉,強制當成新資料處理:

```
@PostMapping("/employees")
public Employee addEmployee(@RequestBody Employee theEmployee) {
    // 就算前端傳了 id,也要強制歸零,確保 JPA 執行 INSERT 而非 UPDATE
    theEmployee.setId(0);
    Employee dbEmployee = employeeService.save(theEmployee);
    return dbEmployee;
}
```

這裡要注意 `id` 欄位型別:如果 `Employee` 的 `id` 宣告成基本型別 `int`,只能設成 `0`(`int` 不能是 `null`);如果宣告成包裝類別 `Integer`,就該設成 `null`。JPA/Hibernate 是靠「`id` 是否為 `null`」來判斷要新增還是更新,型別選錯很容

易讓原本想新增的資料被誤判成更新,這是常見陷阱。用 Postman 測試時記得把 Body 設成 raw + JSON,才會自動帶上 `Content-Type: application/json` 這個必要的標頭。

更新員工(PUT):用 `@PutMapping("/employees")`,方法內容單純委派給 `employeeService.save()`(因為 save 本身就同時處理新增跟更新)。測試完除了看 Postman 回應,務必再查一次資料庫確認真的寫進去了。

PUT 的大陷阱——容易把沒傳的欄位洗成空值:PUT 的語意是「整包資源替換」,如果請求只帶了 `id` 和 `email` 兩個欄位,`firstName`、`lastName` 沒帶到,PUT 會把它們視為缺失,結果資料庫裡這兩欄就被寫成 `null`,原本的資料就這樣不見了。可以想成搬家公司說好「整屋家具原封不動照這張清單擺」,結果清單上漏寫的東西一律當作「不要了」直接丟掉——所以只想改一個欄位卻用 PUT,等於拿著不完整的清單去讓對方全盤替換,非常危險。

PATCH 才是正確的部分更新方案:PATCH 的語意是「只改我有講的欄位,其他保持原樣」。除了避免誤刪資料,還更省頻寬(只傳有變動的欄位),也支援一次改多個欄位。慣例上 PATCH 的資源 ID 放在路徑變數(如 `/api/employees/5`),而不是放進 Body。

實作 PATCH 分三步:

1. 注入 Jackson 的 `JsonMapper`(Spring Boot 已預先配置好,建構子注入即可用),它能把部分更新的 JSON 合併進既有物件。
2. 用 `@PatchMapping("/employees/{employeeId}")`,搭配 `@RequestBody Map<String, Object> patchPayload` 接收酬載——用 `Map` 而不是直接綁定成 `Employee`,是因為 PATCH 請求本來就只會帶部分欄位,用 `Map` 才能彈性地只拿到真正有傳的那幾個 key,不會因為缺欄位而把物件其他欄位覆蓋成 `null`。
3. 把酬載套用到既有物件上再存回資料庫:

```
@PatchMapping("/employees/{employeeId}")
public Employee patchEmployee(@PathVariable int employeeId,
                              @RequestBody Map<String, Object>
                              patchPayload) {

    Employee tempEmployee = employeeService.findById(employeeId);
    if (tempEmployee == null) {
        throw new RuntimeException("Employee id not found - " +
employeeId);
    }

    // 安全性檢查:禁止透過 PATCH 修改主鍵
    if (patchPayload.containsKey("id")) {
        throw new RuntimeException("Employee id not allowed in request body
- " + employeeId);
    }

    Employee patchedEmployee = jsonMapper.updateValue(tempEmployee,
patchPayload);
    Employee dbEmployee = employeeService.save(patchedEmployee);
    return dbEmployee;
}
```

有兩個檢查不能省:找不到員工要丟 `RuntimeException`;酬載裡如果夾帶了 `id` 也要擋下來丟例外,避免客戶端透過部分更新偷改主鍵,破壞資料完整性。`jsonMapper.updateValue()` 只在記憶體裡產生更新後的物件,真

正寫進資料庫還是得呼叫 `employeeService.save()`。若遇到更複雜的需求(巢狀 JSON、陣列操作、欄位搬移等),可以參考 RFC 6902(JSON Patch)、RFC 7386(JSON Merge Patch)等標準規範或對應的開源函式庫,但一般情境下 `JsonMapper` 的覆蓋方式已經夠用。

刪除員工(DELETE):用 `@DeleteMapping("/employees/{employeeId}")`,一樣先查有沒有這個人,沒有就丟例外;有的話呼叫 `employeeService.deleteById(employeeId)`,最後回傳一句確認訊息如 `"Deleted employee id - " + employeeId`。用 Postman 測試 DELETE 時 Body 要設成 `none`,因為刪除操作不需要帶資料。

35. Spring Data JPA 簡介

前面手刻的 DAO 有個明顯痛點:如果系統除了 `Employee` 還要管理 `Customer`、`Student`、`Product` 等其他實體,幾乎得把同樣的程式碼(介面 + `EntityManager` 查詢邏輯)複製貼上一遍又一遍。仔細比較會發現,不同實體的 DAO 程式碼幾乎一模一樣,唯一差異只有「實體類型」(如 `Employee.class`)和「主鍵類型」(如 `int`)這兩個變數而已。這種高度重複的樣板程式碼(boilerplate),正是 Spring Data JPA 要解決的問題。

Spring Data JPA 的核心想法很像蓋房子用預鑄模板:你只要告訴系統「這是什麼類型的房間(實體)」「鑰匙長什麼樣(主鍵型別)」,系統就自動幫你把手電、門窗這些基礎建設(CRUD 方法)全部裝好,不用每次都從零蓋一間。具體做法是建立一個介面去繼承 `JpaRepository<實體類型, 主鍵類型>`:

```
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {  
    // 完全不用寫任何程式碼!  
}
```

光是這樣繼承,就自動「免費」擁有 `findAll()`、`findById()`、`save()`、`deleteById()` 等一系列常用方法,完全不需要另外寫實作類別(`Impl`)。對照一下:沒有 Spring Data JPA 之前,一個實體要兩個檔案(介面 + 實作類別),CRUD 邏輯往往要 30 行以上;用了之後只要一個檔案、大概 3 行程式碼,官方說法是能省下超過七成的開發量。

實際整合到 Service 層時,把原本注入 `EmployeeDAO` 的地方改成注入 `EmployeeRepository`,方法內容直接委派:

```
@Service  
public class EmployeeServiceImpl implements EmployeeService {  
    private EmployeeRepository employeeRepository;  
  
    @Autowired  
    public EmployeeServiceImpl(EmployeeRepository theEmployeeRepository) {  
        this.employeeRepository = theEmployeeRepository;  
    }  
  
    @Override  
    public List<Employee> findAll() {  
        return employeeRepository.findAll();  
    }  
}
```

因為 `JpaRepository` 內建已經處理好事務,Service 方法上原本手動加的 `@Transactional` 也可以拿掉了。

改用 `JpaRepository` 後最容易踩到的坑:`findById()` 回傳的不是 `Employee`,而是 `Optional<Employee>`,直接寫 `return employeeRepository.findById(theId);` 會編譯錯誤(型別不匹配)。`Optional` 是 Java 8 引入的容器,用來取代傳統的 `null` 檢查,常用方法有 `isPresent()` (是否有值)、`get()` (取值,沒值會拋例外)、`orElse()` (沒值時給預設值)、`orElseThrow()` (沒值時丟指定例外)。正確處理方式是先判斷 `isPresent()`,有值才 `get()` 取出,沒值的話與其讓程式帶著潛在的 `null` 往下跑(容易在後面某處炸出 `NullPointerException`),不如直接丟出 `RuntimeException`,讓錯誤在第一時間就清楚浮現:

```
@Override
public Employee findById(int theId) {
    Optional<Employee> result = employeeRepository.findById(theId);
    Employee theEmployee = null;

    if (result.isPresent()) {
        theEmployee = result.get();
    } else {
        throw new RuntimeException("Did not find employee id - " + theId);
    }

    return theEmployee;
}
```

除了開箱即用的基礎 CRUD, Spring Data JPA 之後還能透過 JPQL 寫自訂查詢、用 Query DSL 做更結構化的強型別查詢,或定義符合命名慣例的自訂方法,應付更複雜的查詢需求。